Решение задач проектирования с применением паттернов

1. Построение концептуальной модели

Объектно-ориентированные программы состоят из *объектов*. Объект сочетает *данные* и процедуры для их обработки (*методы*). Объект выполняет операцию, когда получает запрос (или сообщение) от клиента.

Отправка запроса — это единственный способ заставить объект выполнить *операцию*. А выполнение *операции* — единственный способ изменить внутреннее состояние объекта.

Из-за этих двух ограничений *внутреннее состояние объекта инкапсулировано*: к нему нельзя обратиться напрямую, а его представление невидимо за пределами объекта.

Концептуальные модели оперируют понятиями предметной области, атрибутами этих понятий и отношениями между ними.

Понятию в предметной области могут соответствовать как материальные предметы, так и абстракции, которые применяют специалисты предметной области



В качестве *атрибутов* представляют некоторые, существенные с точки зрения решаемой задачи характеристики объектов, например идентифицирующие значения (имя, номер)

Если некоторый объект ***X*** в реальном мире *не* является числом или текстом, то это, скорее всего *понятие*. В противном случае - это *атрибут*

1. Построение концептуальной модели

Методологии ООП в декомпозиции предметной области:

- сформулировать задачу письменно, выделить из получившейся фразы существительные и глаголы, после чего создать соответствующие классы и операции;
- проанализировать отношения и разделение обязанностей в системе;
- построить модель реального мира и перенести выявленные при анализе объекты в проект;

Категория понятий-кандидатов	Примеры
Физические или материальные объекты	Самолет - как целое
Спецификации, элементы дизайна или описания объекта - сохраняются в системе даже при отсутствии объектов	Цвет, спецификация товара
Место	Аэропорт, город
Роль человека	Продавец, покупатель, преподаватель
Контейнеры других объектов	Самолет - как совокупность частей, каталог - как совокупность описаний
Содержимое контейнеров	Часть, элемент
Системы, внешние по отношению к разрабатываемой	Система бронирования билетов
Абстрактные понятия	Летательный аппарат
Организации	Фирма, предприятие, НИИ
События	Встреча, покупка билета
Процессы и их части*	Покупка билета, оплата стоимости
Правила и политика	Правила аннулирования заказа билета
Записи финансовой, трудовой, юридической и другой деятельности, руководства, книги	Чек, книга учета, должностная инструкция

2. Определение степени детализации объекта

Размеры и число объектов могут изменяться в широком диапазоне. С помощью программных объектов можно представить любые сущности реального мира.

Какова глубина абстракции?

Паттерн **фасад** (Facade) показывает, как представить в виде объекта целые подсистемы

Паттерн **приспособленец (**Flyweight**)** — как поддерживать большое число объектов при высокой степени детализации

Абстрактная фабрика (Abstract Factory) и **строитель** (Builder) описывают объекты, единственной целью которых является создание других объектов

Посетитель (Visitor) и **команда** (Command) - объекты, отвечающие за реализацию запроса к другому объекту или группе

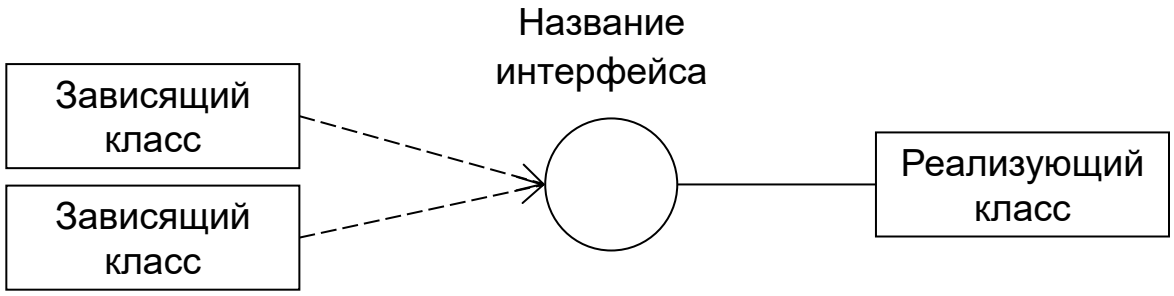
3. Определение интерфейсов объекта

Для любой операции, объявляемой объектом, должны быть заданы: *имя операции, объекты, передаваемые в качестве параметров, и значение, возвращаемое операцией*.

Эту триаду называют **сигнатурой операции**.

Множество сигнатур всех определенных для объекта операций называется **интерфейсом** этого объекта.

Интерфейс описывает все множество запросов, которые можно отправить объекту. Любой запрос, сигнатура которого входит в интерфейс объекта, может быть ему отправлен.



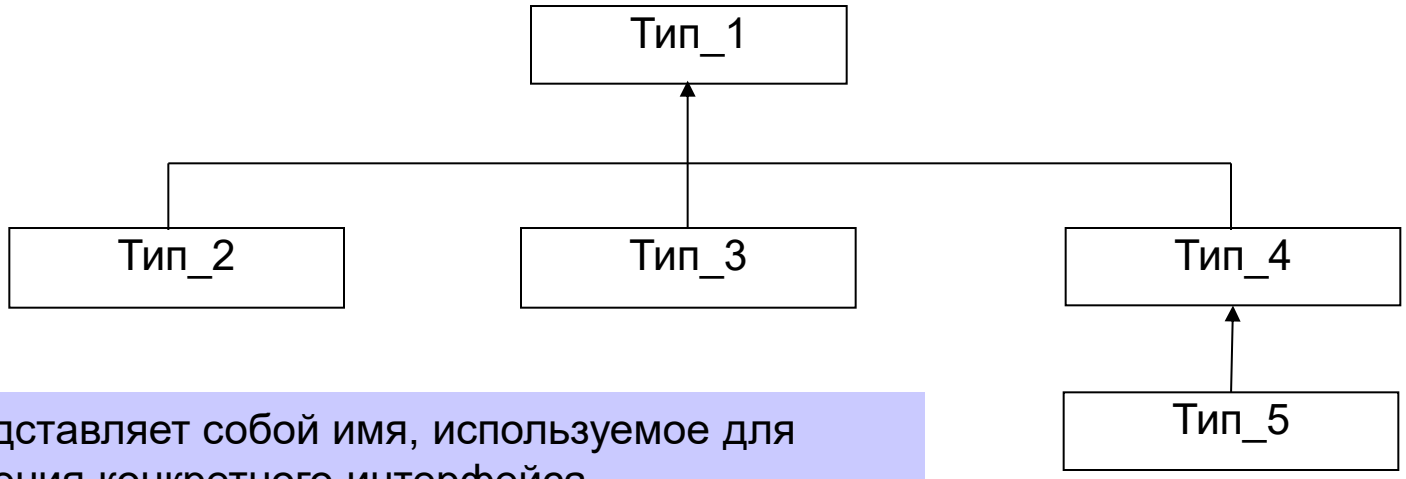
Все взаимодействие с объектами осуществляется через их *интерфейсы*.

Никакого способа получить информацию об объекте или заставить его что-то сделать в обход интерфейса не существует (no friend!).

Интерфейс объекта ничего не говорит о его реализации; разные объекты вправе реализовывать сходные запросы совершенно по-разному.

Это означает, что два объекта с различными реализациями могут иметь одинаковые интерфейсы.

Наследование интерфейсов



Тип представляет собой имя, используемое для обозначения конкретного интерфейса (*разделение интерфейсов, множественное наследование, супертип, подтип*)

Когда объекту посылается запрос, то операция, которую он выполнит, зависит как от *запроса*, так и от *объекта-адресата*.

Разные объекты, поддерживающие одинаковые интерфейсы, могут выполнять в ответ на такие запросы разные операции.

Ассоциирование запроса с объектом и одной из его операций во время выполнения называется ***динамическим связыванием***.

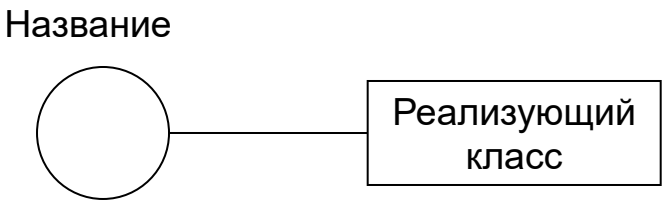
3. Определение интерфейсов объекта

Динамическое связывание означает, что отправка некоторого запроса *не определяет никакой конкретной реализации до момента выполнения*.

Возможно написать программу, рассчитанную на объект с конкретным интерфейсом, точно зная, что любой объект с подходящим интерфейсом сможет принять этот запрос.

Динамическое связывание позволяет во время выполнения подставить вместо одного объекта другой, если он имеет идентичный интерфейс.

Такая взаимозаменяемость называется ***полиморфизмом*** и является важнейшей особенностью объектно-ориентированных систем.

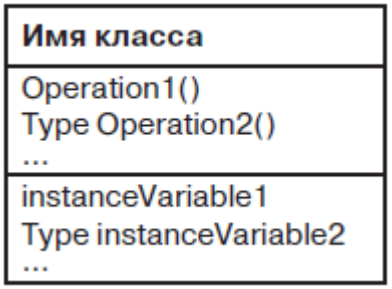


Паттерны проектирования позволяют определять интерфейсы посредством задания их основных элементов и того, какие данные можно передавать через интерфейс.

4. Определение реализации объектов

Реализация объекта определяется его *классом*.

Класс определяет внутренние данные объекта и его представление, а также операции, которые объект может выполнять.



Объекты создаются посредством создания *экземпляров* класса.

В процессе создания экземпляров выделяется память для внутренних данных объекта (переменных экземпляра), а с этими данными связываются операции.

Создавая экземпляры *одного класса*, можно создать много *сходных объектов*.

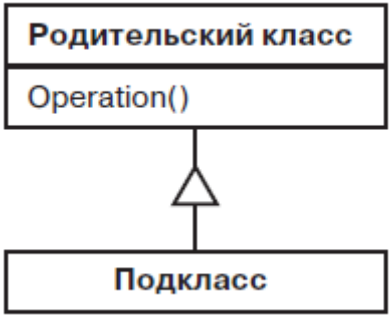


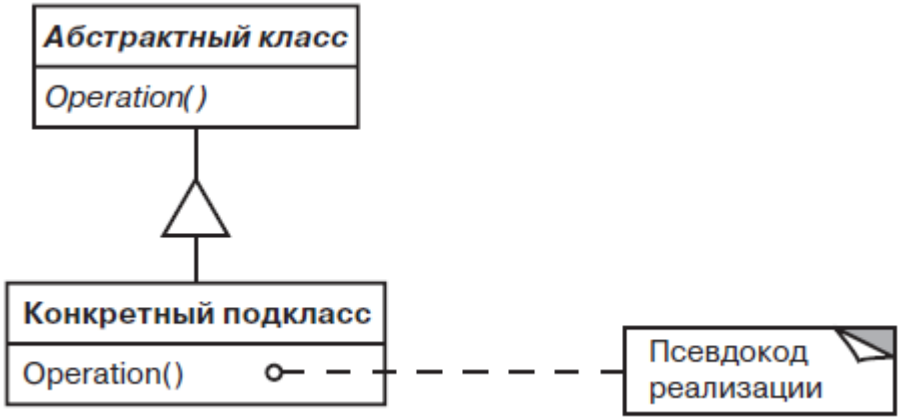
4. Определение реализации объектов

Новые классы могут определяться в контексте существующих с использованием ***наследования*** классов.

Если подкласс наследует родительский класс, то он включает определения всех данных и операций, определенных в родительском классе.

Объекты, являющиеся экземплярами подкласса, будут содержать все данные, определенные как в самом подклассе, так и во всех





Класс называется ***абстрактным***, если его единственное назначение — определить общий интерфейс для всех своих подклассов.

Абстрактный класс делегирует реализацию всех или части своих операций подклассам, поэтому у него не может быть экземпляров.

Операции, объявленные, но не реализованные в абстрактном классе, называются *абстрактными*. Класс, не являющийся абстрактным, называется *конкретным*.

Подклассы могут уточнять или переопределять поведение своих предков.

Наследник может заместить операцию, определенную в *родительском* классе (*private*).

Замещение дает подклассам возможность обрабатывать запросы, адресованные родительским классам. Наследование позволяет определять новые классы, расширяя возможности старых.



Примесью (*mix-in class*) называется класс, назначение которого - предоставить дополнительный интерфейс или функциональность другим классам.

Он отчасти похож на *абстрактные классы* в том смысле, что не предполагает непосредственного создания экземпляров.

Для работы с примесями необходимо *множественное* наследование (иногда *включение*)

5. Наследование класса и наследование интерфейса

В некоторых языках программирования существует различие между *классом* объекта и его *типом (Haskell).*

Класс объекта определяет реализацию объекта (т.е. *внутреннее состояние)* и *реализацию операций* объекта.

Тип относится только к *интерфейсу* объекта — *множеству запросов*, на которые объект способен ответить.

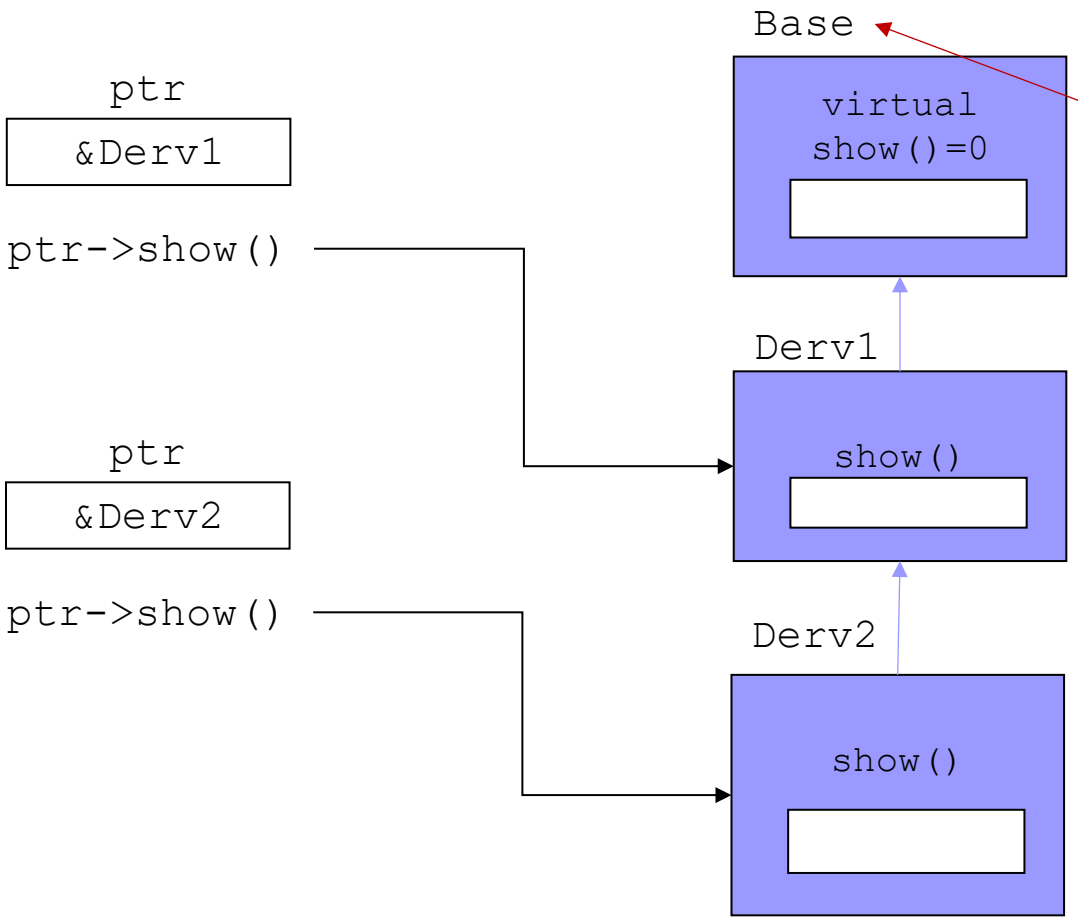
В C++, C# такое разделение отсутствует. Под наследованием понимается одновременно наследование *интерфейса* и *реализации*.

Стандартный способ реализации наследования интерфейса в C++ — это ***public*** наследование абстрактному классу.

Наследование классов может моделироваться посредством ***private*** наследования.

5. Наследование класса и наследование интерфейса

При правильном использовании *наследования* - все классы, производные от некоторого абстрактного класса, будут обладать его интерфейсом



У манипулирования объектами строго через интерфейс ***абстрактного класса*** есть два преимущества:

- клиенту не нужно располагать информацией о конкретных типах объектов, которыми он пользуется, при условии что все они имеют ожидаемый клиентом интерфейс;
- клиенту необязательно «знать» о классах, с помощью которых реализованы объекты. Клиенту известно только об абстрактном классе (или классах), определяющих интерфейс.

```
dynamic_cast<тип_к_которому_приводим>(приводимый_объект)
```

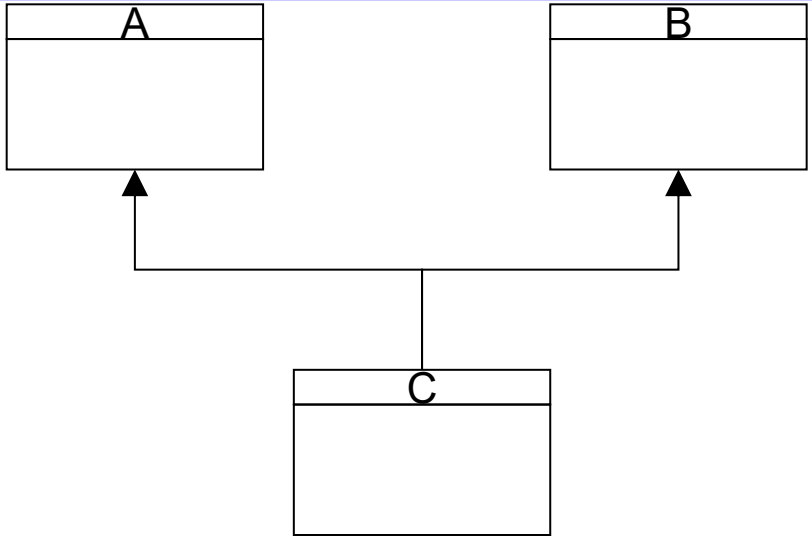
03_virt_pure.cpp
07_dyncast2.cpp

6. Механизмы повторного использования

6.1 Наследование и композиция

Наследование - повторное использование за счет *порождения подкласса* называют еще повторным использованием по принципу *прозрачного ящика (white box reuse*). Такой термин подчеркивает, что внутреннее устройство родительских классов часто видимо подклассам (*public*).

```
class A
{
};
class B
{
};
class C : public A, public B
{
};
```



Наследование класса определяется *статически* на этапе компиляции. В случае наследования классов упрощается задача модификации существующей реализации. Если подкласс замещает лишь некоторые операции, то могут оказаться затронутыми и остальные унаследованные операции (при условии что они вызывают замещенные).

Проблемы наследования:

- унаследованную от родителя реализацию не удастся изменить во время выполнения программы, поскольку наследование определяется на этапе компиляции;
- поскольку подклассу доступны детали реализации родительского класса, то часто говорят, что наследование нарушает инкапсуляцию;
- реализации подкласса и родительского класса связываются настолько тесно, что любые изменения последней требуют изменять и реализацию подкласса.

Зависимость от реализации может повлечь за собой проблемы при попытке повторного использования подкласса.

Если хотя бы один аспект унаследованной реализации непригоден для новой предметной области, то приходится переписывать родительский класс или заменять его чем-то более подходящим.

Такая зависимость ограничивает гибкость и возможности повторного использования.

С проблемой можно справиться, если наследовать только *абстрактным классам*.

Композиция объектов — альтернатива наследованию класса. В этом случае новая, более сложная функциональность получается путем объединения или композиции объектов.
Такой способ называют повторным использованием по принципу *черного ящика (blackbox reuse)*, поскольку детали внутреннего устройства объектов остаются скрытыми.

```
class books
{};
class staff
{};
class lib
{
  books objB;
  staff objS;
};
```



Композиция объектов определяется *динамически* во время выполнения за счет того, что объекты получают ссылки на другие объекты

Поскольку доступ к объектам осуществляется только через их *интерфейсы*, инкапсуляция не нарушается. Во время выполнения программы любой объект можно заменить другим, лишь бы он имел тот же тип.

Так как реализация объекта пишется прежде всего в категориях его интерфейсов, то зависимость от реализации резко снижается.

6. Механизмы повторного использования

Применяя композицию объектов вместо наследования классов, получаем инкапсулированные классы, выполняющие свои специфические задачи.

Классы и их иерархии остаются небольшими, и вероятность их разрастания до неуправляемых размеров невелика.

НО, дизайн, основанный на композиции, будет содержать больше объектов (хотя число классов, возможно, уменьшится), а поведение системы начнет зависеть от их взаимодействия, тогда как при другом подходе оно было бы определено в одном классе.

Рекомендовано наследование:

- для моделирования статических отношений;
- чтобы использовать как интерфейс, так и реализацию базового класса;
- для предоставления примесей (**mix-in**) нескольким несвязанным классам, когда может существовать только одна реализация этой функции.

Рекомендована композиция:

- для создания компонентов, которые могут повторно использоваться несколькими классами;
- для реализации групп поведений и политик, которые можно применять взаимозаменяемо к другим классам для настройки их поведения;
- для изменения поведения во время выполнения, не затрагивая существующие классы.

6.2 Делегирование

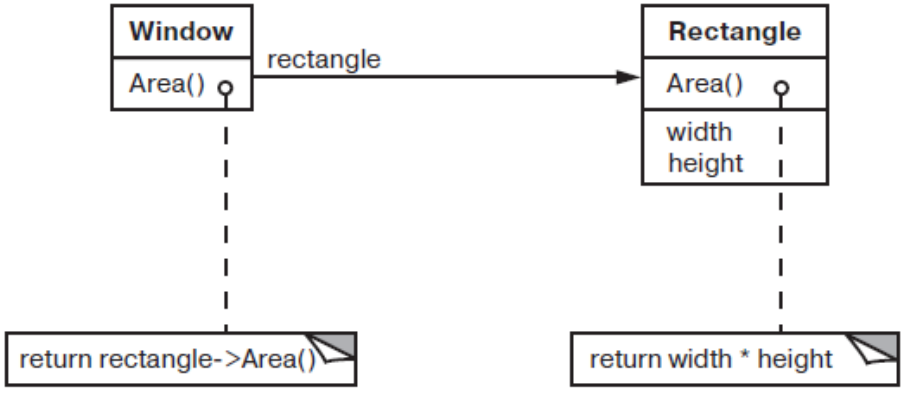
При делегировании в процессе обработки запроса задействованы два объекта: *получатель* поручает выполнение операций другому объекту — *уполномоченному (делегату).*

Примерно так же *базовый класс* делегирует ответственность своему *производному* классу.

- Унаследованная операция всегда может обратиться к объекту-получателю через переменную класса.
- Чтобы достичь того же эффекта для делегирования, *получатель* передает указатель на самого себя соответствующему объекту, дабы при выполнении делегированной операции последний мог обратиться к непосредственному адресату запроса.

НАПРИМЕР, Вместо того чтобы делать класс ***Window*** (*окно*) подклассом класса ***Rectangle*** (*прямоугольник*) — если окно является прямоугольником, —можно воспользоваться внутри ***Window*** поведением класса ***Rectangle***, поместив в класс ***Window*** переменную экземпляра типа ***Rectangle*** и делегируя ей операции, специфические для прямоугольников. Другими словами, окно не является прямоугольником, а *содержит его*.

Таким образом класс ***Window*** может явно перенаправлять запросы своей переменной ***Rectangle***, а не наследовать ее операции.



deleg_compoz.cpp

Главное **достоинство** делегирования в том, что оно упрощает композицию поведений во время выполнения (*динамически, позднее связывание*)

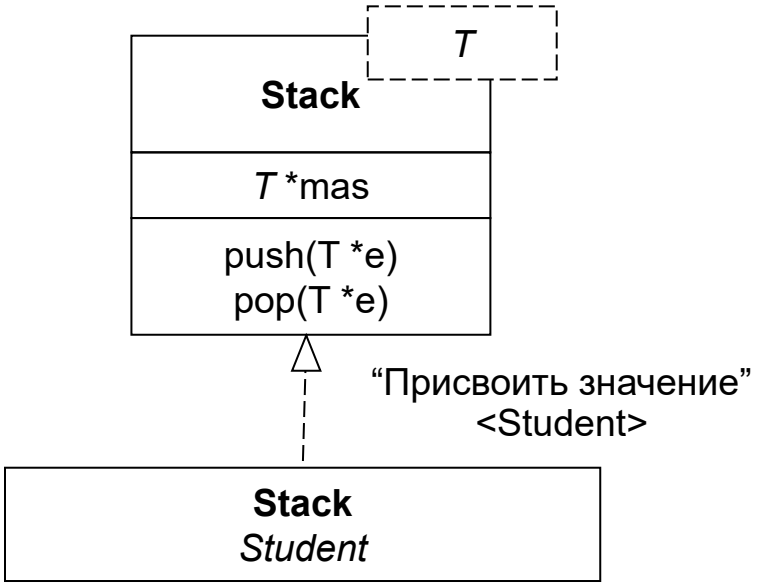
Недостаток - затраты времени на выполнение позднего связывания

Делегирование — особый случай композиции.
Оно показывает, что *наследование* как механизм повторного использования всегда можно заменить *композицией*.

6.3 Наследование и параметризированные типы

Еще один метод повторного использования имеющейся функциональности — это применение *параметризованных типов*, известных также как ***шаблоны***.

Данная техника позволяет определить тип, не задавая типы, которые он использует. Отсутствующие типы передаются в параметрах в точке использования.



- *Композиция* объектов позволяет изменять поведение во время выполнения, но для этого требуются косвенные вызовы, что снижает эффективность.
- *Наследование* разрешает предоставить реализацию по умолчанию, которую можно замещать в подклассах.
- *Параметризованные* типы позволяют изменять типы, используемые классом.
- *Ни наследование, ни параметризованные* типы не могут изменяться во время выполнения.

tempstack2.cpp

6.4 Агрегирование и ассоциация

Структура ОО программы

Времени компиляции (статическая)	Времени выполнения (динамическая)

Ассоциация - статическая, не зависящая от времени, связь между экземплярами классов (отношение типа «использует»).

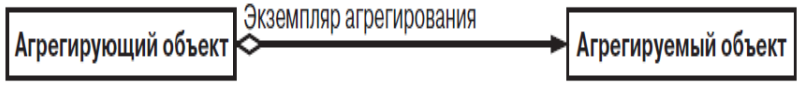
Объекты могут запрашивать друг у друга операции, но они не несут никакой ответственности друг за друга.

Ассоциация — это более слабое отношение, чем *агрегирование*; оно предполагает гораздо менее тесную связь между объектами.



Агрегирование подразумевает, что один объект владеет другим или несет за него ответственность (*композиция* – агрегирование по значению).

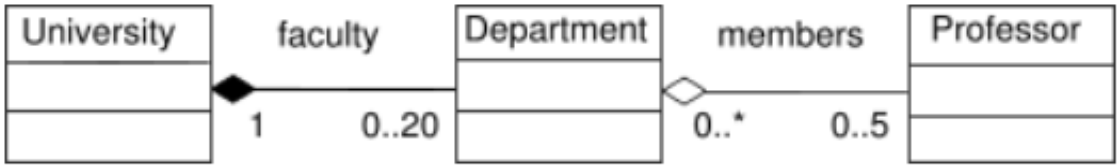
Объект *содержит* другой объект или *является его частью*.



6. Механизмы повторного использования

Агрегация: *профессора — факультеты*, профессора остаются жить после разрушения факультета

Композиция: *университет — факультеты*, факультеты без университета погибают.



Агрегация (*агрегирование по ссылке*) — отношение *«часть-целое»* между двумя равноправными объектами, когда один объект (*контейнер*) имеет ссылку на другой объект. Оба объекта могут существовать независимо: если *контейнер* будет уничтожен, то его *содержимое* — нет.

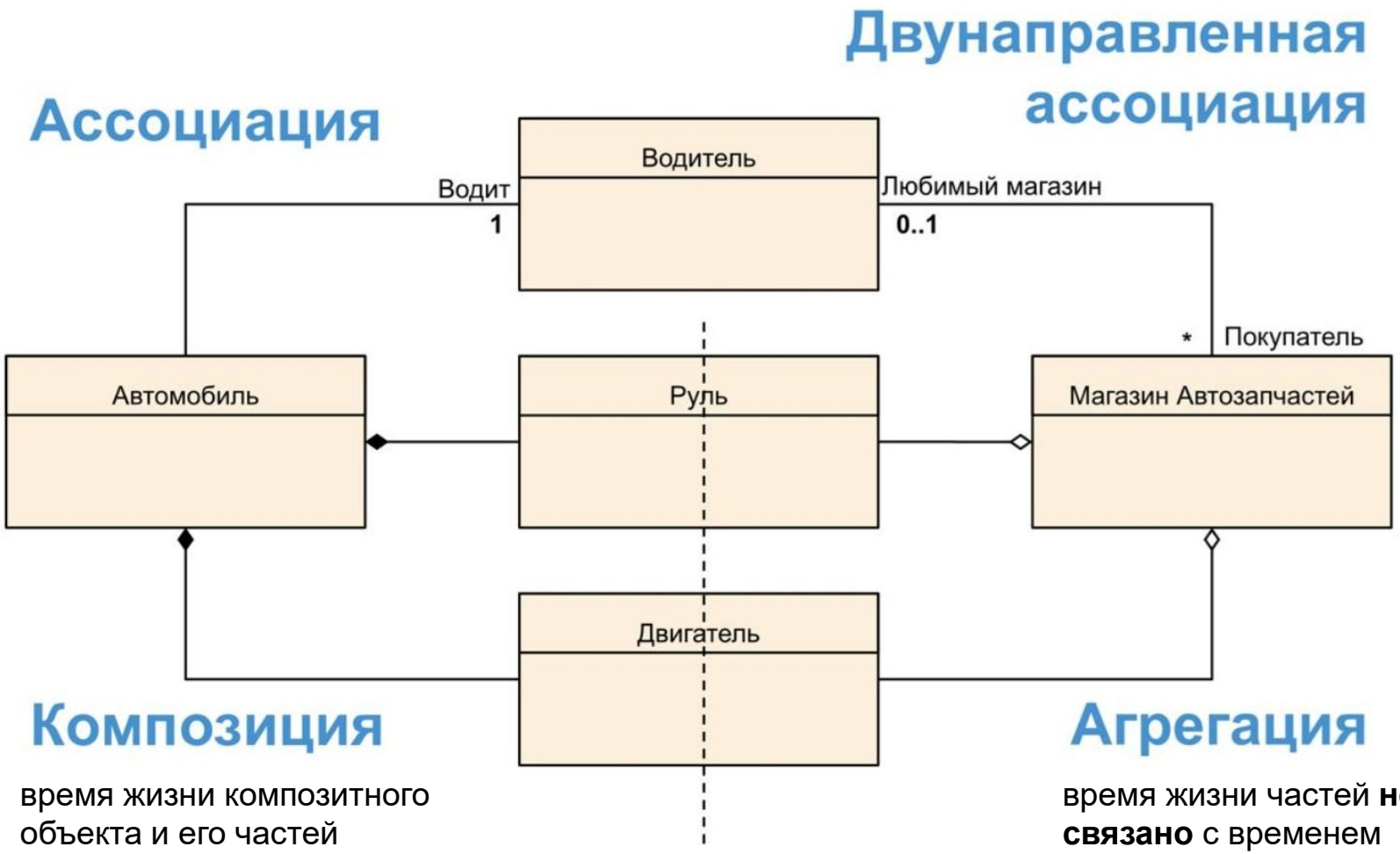
```
class Professor;

class Department
{
  private:
    Professor* members[5];
/*агрегация, т. к. нет оператора
delete*/
};
```

Композиция (*агрегирование по значению*) — более строгий вариант агрегирования, когда включаемый объект может существовать только как часть *контейнера*. Если *контейнер* будет уничтожен, то и включённый объект тоже будет уничтожен.

```
class Department;
class University
{  private:
     Department faculty[20];// композиция
};
//-----------------------------------
class Department;
class University
{  private:
     Department* faculty;// композиция
public:
 University() {faculty =new Department();}
    virtual ~ University()
{delete faculty;} /* композиция, т. к.
объекты faculty будут удалены */};
```

6. Механизмы повторного использования



время жизни композитного объекта и его частей **совпадают**

время жизни частей **не связано** с временем жизни агрегата

7. Проектирование с учетом будущих изменений

Системы должны проектироваться с учетом **их *дальнейшего развития***. Для проектирования системы, устойчивой к таким изменениям, следует предположить, как она будет изменяться на протяжении отведенного ей времени жизни.

Если при проектировании системы не принимать во внимание возможность изменений, то есть вероятность, что в *будущем ее придется полностью перепроектировать.* Это может повлечь за собой переопределение и новую реализацию классов, модификацию клиентов и повторный цикл тестирования. Перепроектирование отражается на многих частях системы, поэтому непредвиденные изменения всегда оказываются дорогостоящими.

Благодаря паттернам систему можно модифицировать с минимальным привлечением зависимостей. Каждый паттерн позволяет изменять некоторый аспект системы ***независимо от всех прочих***, таким образом, она менее подвержена влиянию изменений конкретного вида.

7. Проектирование с учетом будущих изменений

Причины перепроектирования

Причина	Проблема	Решение	Паттерны
При создании объекта явно указывается класс	Задание имени класса привязывает к конкретной реализации, а не к конкретному интерфейсу. Проблемы с изменением объекта в будущем.	Создание объектов косвенно	**абстрактная фабрика, фабричный метод, прототип**
Зависимость от конкретных операций	Задание конкретной операции ограничивает способы выполнения запросов	Не включать запросы в код	**цепочка обязанностей, команда**
Зависимость от аппаратной и программной платформ	Внешние интерфейсы операционной системы и интерфейсы прикладных программ (API) различны на разных программных и аппаратных платформах	Ограничение платформенных зависимостей	**абстрактная фабрика, мост**

7. Проектирование с учетом будущих изменений

Причина	Проблема	Решение	Паттерны
Зависимость от представления или реализации объекта	Если клиент располагает информацией о том, как объект представлен, хранится или реализован, то, возможно, при изменении объекта придется изменять и клиента.	Сокрытие деталей реализации от клиентов снизит вероятность каскадных изменений	**абстрактная фабрика, мост, хранитель, заместитель**
Зависимость от алгоритмов	Во время разработки и последующего использования алгоритмы могут модифицироваться. Зависящие от алгоритмов объекты придется переписывать при каждом изменении алгоритма.	Алгоритмы, которые с большой вероятностью будут изменяться, следует изолировать	**мост, итератор, стратегия, шаблонный метод, посетитель**
Сильная связанность	Сильно связанные между собой классы трудно использовать отдельно, так как они зависят друг от друга. Сильная связанность приводит к появлению монолитных систем, в которых изменения одного класса вызывают необходимость изменения других.	Для реализации слабосвязанных систем в паттернах проектирования применяются такие методы, как *абстрактные связи и разбиение на слои*.	**абстрактная фабрика, мост, цепочка обязанностей, команда, фасад, посредник, наблюдатель**

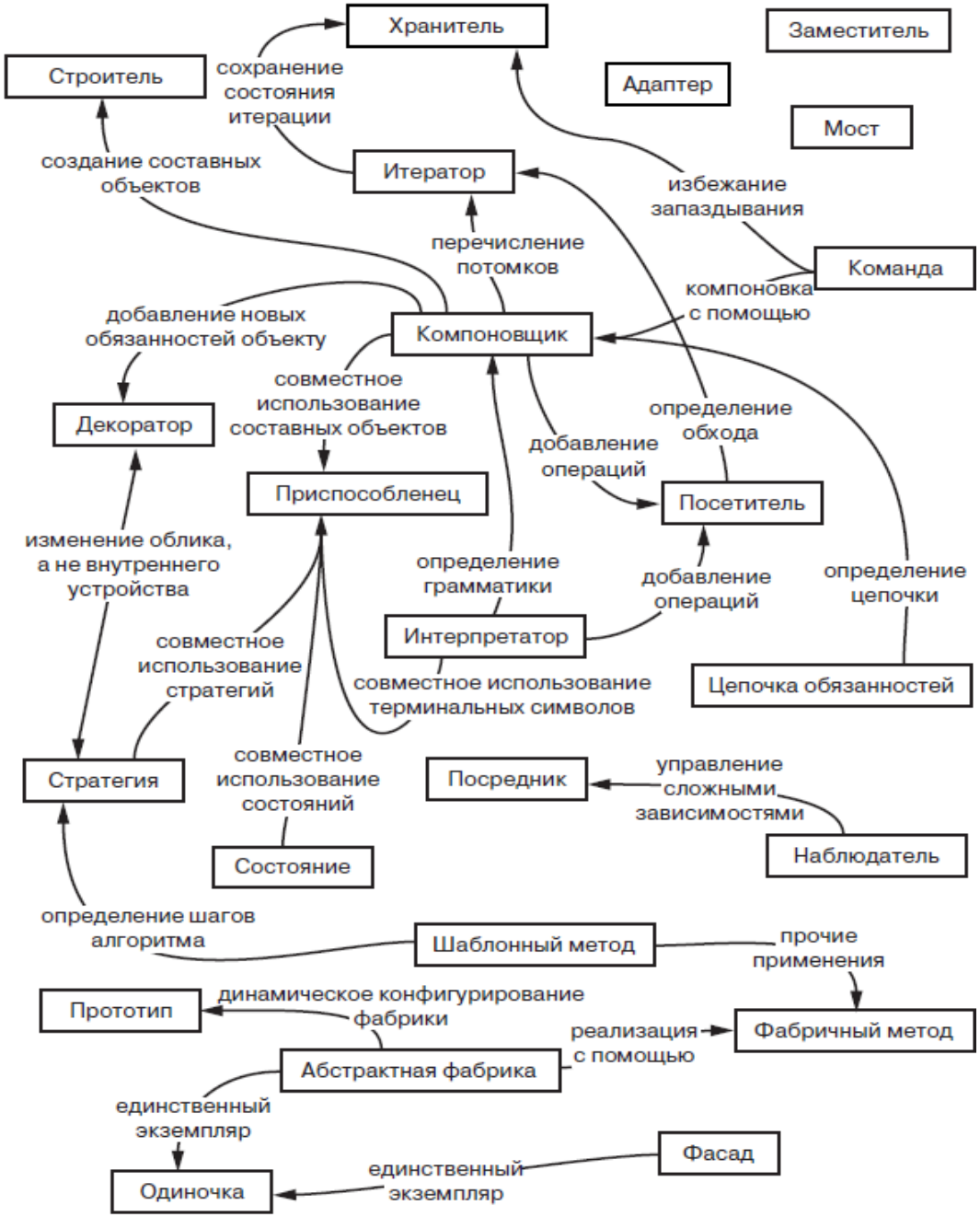
7. Проектирование с учетом будущих изменений

Причина	Проблема	Решение	Паттерны
Расширение функциональности за счет порождения подклассов	Разрастание иерархии классов; рост фиксированных издержек реализации (инициализация, очистка и т. д.); цепочка изменений.	Композиция объектов и делегирование	**мост, цепочка обязанностей, компоновщик, декоратор, наблюдатель, стратегия**
Неудобства при изменении классов	Закрытый исходный код классов (проприетарная коммерческая лицензия). Длинная цепочка изменений классов.	Изолирование классов	**адаптер, декоратор, посетитель**

8. Процедура выбора паттернов проектирования

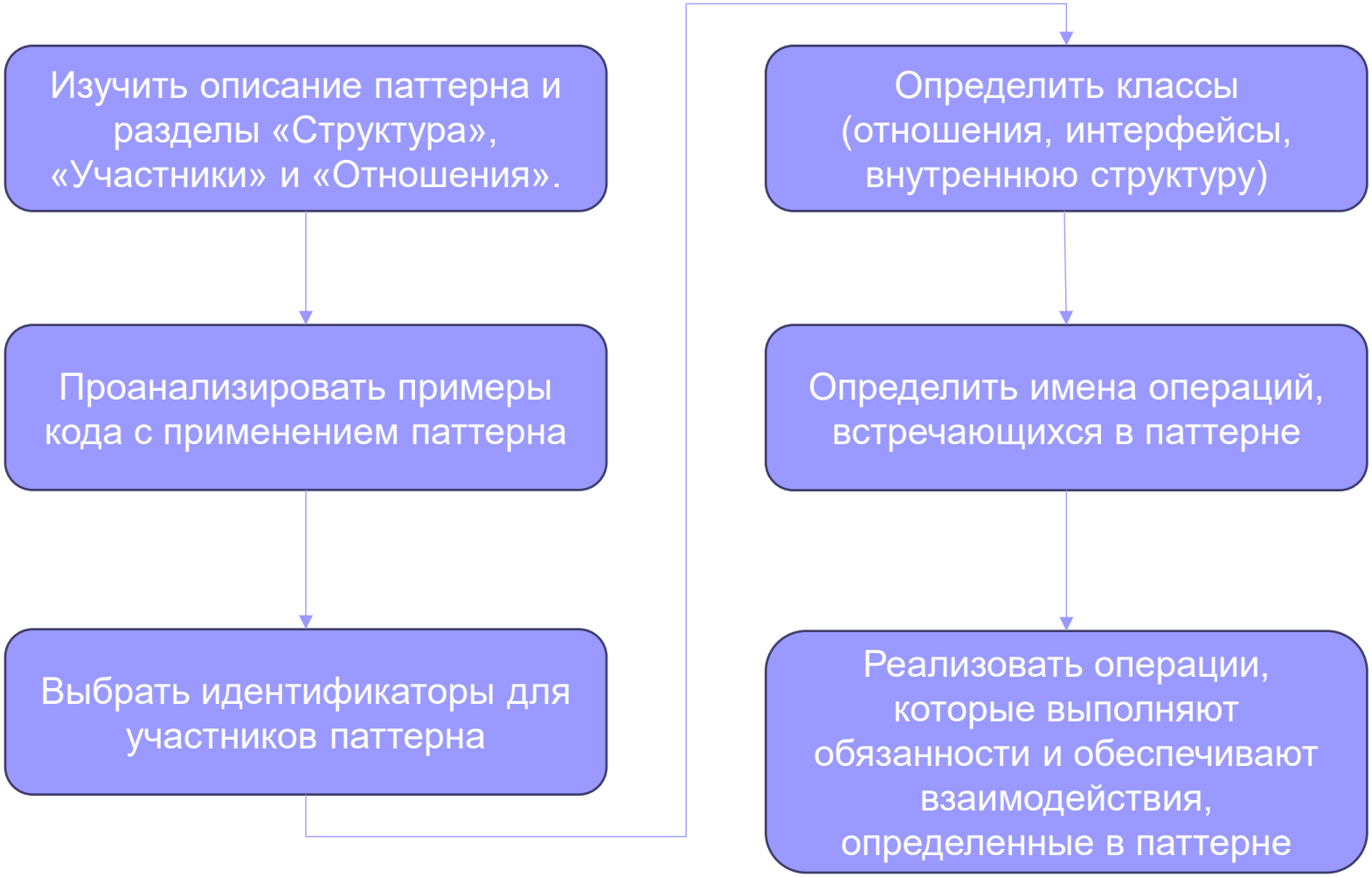


8. Процедура выбора паттернов проектирования



Отношения между паттернами проектирования

9. Схема использования паттернов проектирования



Результаты применения паттернов:
Гибкость vs Усложнение проекта vs Ухудшение производительности